

Exp-03: Multi-Level Cache Design

Workload-Driven Analysis of a Three-Level (L1/L2/L3) Cache Hierarchy

Sudarshan Sudhakar
Roll No: CS23B2007

August 25, 2026

Abstract

This report simulates a three-level cache hierarchy (L1 32KB, L2 256KB, L3 2MB, all with 64B lines, backed by a 64MB / 26-bit DRAM) driven by a single four-phase synthetic workload – embedding lookup, matrix multiplication, streaming activation, and output projection – run as one continuous instruction stream without clearing the caches between phases. Three associativity configurations are compared: the assignment baseline (L1 1-way, L2 2-way, L3 4-way), a “raise everything” configuration (2/4/8-way), and an L1-focused configuration (4/4/8-way) that isolates whether extra L1 associativity specifically matters. All numbers reported below are the direct, unedited output of the accompanying simulator (`cache_sim.py`); every table is generated automatically from that run and \inputed here verbatim. The central finding is that associativity’s effect is highly phase-dependent and can move misses *between* levels rather than eliminate them outright: raising L1 associativity resolves a severe, address-aliasing-induced conflict-miss storm in two of the four phases, but simultaneously starves the higher levels of the very reuse traffic that used to pad their hit rate, making L2 and L3 *miss rate* rise sharply even while total DRAM traffic and overall AMAT both improve. This motivates the report’s final recommendation and its explicit warning against reading hit rate alone as a proxy for performance.

Contents

1	Cache Hierarchy and System Parameters	2
2	Simulator Design	2
3	Workload: Four Phases	3
3.1	Phase 1 – Embedding / Lookup	3
3.2	Phase 2 – Matrix / Tensor Computation	3
3.3	Phase 3 – Activation Processing	4
3.4	Phase 4 – Output / Projection	4
4	Access-Pattern and Locality Analysis (Predictions Before Running)	4
5	Cache Hierarchy Configurations Tested	5
6	Baseline Results	5
7	Experimental Results	6
8	Important Observations	8
8.1	Observation 1 – A genuine, quantifiable conflict-miss storm, fixed by 2-way L1 .	8
8.2	Observation 2 – The same fix moves the miss burden <i>up</i> , not away	8
8.3	Observation 3 – Increasing associativity does not fix a pure capacity problem . .	9

8.4	Observation 4 – A configuration that helps the whole workload can hurt one phase	9
8.5	Observation 5 – Diminishing returns from L1 4-way over 2-way	9
9	Architectural Explanation of These Observations	9
10	Recommended Configuration	10
11	One Surprising Result	10
A	Source Code	11

1 Cache Hierarchy and System Parameters

Parameter	Value
Main memory (DRAM) capacity	64 MB
Address size	26 bits (byte-addressable)
DRAM latency	100 cycles
Cache line size (all levels)	64 B (fixed throughout the experiment)

Table 1: Main memory parameters

Level	Capacity	Line size	Baseline mapping	Latency
L1	32 KB	64 B	Direct mapped (1-way)	1 cycle
L2	256 KB	64 B	2-way	5 cycles
L3	2 MB	64 B	4-way	20 cycles

Table 2: Baseline cache hierarchy

Every request is looked up strictly in CPU $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ DRAM order (Section 2 describes the simulator’s enforcement of this). When a block is found at L2, it is filled into L1; when found at L3, it is filled into both L2 and L1; when it must be fetched from DRAM, it is filled into all three levels. **Latency model / stated assumption:** the simulator uses a cumulative, serial-probe latency model – an access that misses in L1 and hits in L2 costs $L1_{\text{lat}} + L2_{\text{lat}} = 6$ cycles, one that falls through to DRAM costs $1 + 5 + 20 + 100 = 126$ cycles, and so on. This is a simplifying but standard assumption for a simulator at this level of abstraction, and is applied identically to every configuration, so it does not bias the comparison between configurations.

2 Simulator Design

The entire simulator, workload generator, experiment driver, and result/plot writer live in a single file, `cache_sim.py`, organised into clearly separated sections:

- **Cache** – a generic N -way set-associative cache with true LRU replacement. `access(addr)` is a genuine demand access (counted as hit/miss); `fill(addr)` inserts a line without counting a hit/miss, modelling the “fill upper levels” step of the hierarchy diagram.
- **MultiLevelCache** – wires three **Cache** instances into the mandated CPU $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ DRAM order, filling upper levels on every hit below L1 and accumulating hit/miss counts, DRAM accesses, and total cycles.

- **Phase address generators** – four Python generators (`phase1_addresses .. phase4_addresses`) that lazily yield the byte-address stream for each workload phase, so tens of millions of addresses never need to be materialised as a list in memory.
- **Experiment driver** (`run_config, main`) – runs the complete four-phase workload once per configuration through a single, never-cleared `MultiLevelCache` instance, snapshotting hit/miss counters before and after every phase to derive phase-wise deltas.
- **Output** – JSON (`raw_results.json`), two CSV tables matching the exact layout requested in the assignment (`phase_wise_results.csv`, `config_summary.csv`), an append-only `master_ledger.csv` (every run is logged with a timestamp, never overwritten), a `tables.tex` fragment (auto-generated LaTeX tables, \inputed directly into this report – see Sections 5–6), and four matplotlib plots.

All addresses across all four phases live in the same 26-bit / 64 MB address space – there is no other address space available on this system. Phase 1’s embedding table alone is exactly 64 MB ($1,048,576$ entries $\times$ 64 B), i.e. it occupies the *entire* address space by construction (Section 3), so Phases 2–4 necessarily reuse addresses Phase 1 already touched, in the same way a real allocator reuses heap space once the embedding stage no longer needs it. Within each phase, arrays are placed at fixed, non-overlapping offsets so the phase-internal access pattern is unambiguous. Where a variable is loop-invariant across a loop’s innermost index (e.g. `C[i][j]` across the k -loop in Phase 2, or `Y[b][o]` across the k -loop in Phase 4), the simulator models the standard register-allocated behaviour: the accumulator lives in a register across the inner loop and is written back to memory once, rather than issuing a redundant load/store on every iteration. This is exactly what a real compiler emits for this code and keeps the address-stream size tractable without altering any of the spatial/strided/temporal properties the experiment is actually about (those live entirely in the A/B and X/W traffic).

3 Workload: Four Phases

All four phases run back-to-back as a single instruction stream; the cache is *not* cleared between phases, so each phase inherits whatever state the previous phase left behind.

3.1 Phase 1 – Embedding / Lookup

A $1,048,576 \times 16$ embedding table (4 B per value) = exactly 64 MB, so each entry is exactly one 64 B cache line. 50,000 lookups are generated from a fixed-seed stream that mixes three components per the spec’s own example (17, 2048, 91, 17, 8192, 2048, 503, 91, ...): a 5% chance of an immediate repeat of the previous index, a 35% chance of drawing from a small 64-entry “hot” pool (temporal reuse), and 60% uniformly random draws across all 1,048,576 entries (irregular, weak-spatial-locality traffic). Each lookup issues 16 four-byte reads (one per value in the entry).

3.2 Phase 2 – Matrix / Tensor Computation

$C = A \times B$, all 256×256 (256 KB each, 768 KB total), ijk loop order. `A[i][k]` is read sequentially as k advances (strong spatial locality). `B[k][j]` is read with a stride of $256 \times 4 = 1024$ B as k advances (poor spatial locality – one cache line touched per access, only 4 of its 64 bytes used), but the *same* 256 addresses of a given column are revisited on every pass of the outer i -loop (256 times total) – strong temporal locality, *if* the cache can hold the working set across that gap. `C[i][j]` is accumulated in-register (Section 2) and written once per (i, j) .

3.3 Phase 3 – Activation Processing

$\text{output}[i] = \text{activation}(\text{input}[i])$ over $N = 1,048,576$ elements (4 MB input + 4 MB output = 8 MB, exceeding even L3). Purely sequential/streaming: every address is touched exactly once, so spatial locality is strong (15 hits after every 16-word line’s first miss) but temporal locality is essentially absent.

3.4 Phase 4 – Output / Projection

A batched linear projection $Y = XW$, with X : 512×128 (256 KB), W : 128×32 (16 KB), Y : 512×32 (64 KB). $X[b][:]$ is reused across all 32 output columns for a given batch row (temporal reuse); $W[o][:]$ is reused across all 512 batch rows (heavy temporal reuse of a small, easily-cacheable array); $Y[b][o]$ is written once per (b, o) at a stride of $32 \times 4 = 128$ B (strided, essentially no reuse within the phase).

4 Access-Pattern and Locality Analysis (Predictions Before Running)

Phase	Dominant pattern	Spatial locality	Temporal locality	Expected hierarchy behaviour
Embedding	Irregular / random, with a hot-pool component	Weak (each 64 B entry is one line – no locality <i>between</i> distinct lookups)	Some (hot pool, occasional immediate repeat)	High cache pressure; most first-touches should miss all the way to DRAM since the table (64 MB) vastly exceeds every level; hot-pool hits should mostly land in L1/L2.
Matrix	Sequential (A) + strided (B) + repeated (B across i)	Strong for A, weak for B	Strong for B <i>if</i> it survives in cache across the i -loop	L1 (32 KB) is far smaller than B (256 KB) alone, so L1 should show high miss rate regardless of associativity (a capacity problem); L2/L3 (256 KB / 2 MB) are large enough to hold B or all of A+B+C, so associativity should matter there.
Activation	Pure streaming	Strong (line reuse only)	Essentially none	Every line is compulsory; increasing cache size or associativity should not reduce DRAM traffic much, since nothing is ever revisited long-range – a good test of “does spatial locality alone justify caching.”
Projection	Sequential (X,W) + reused (W heavily, X moderately) + strided (Y)	Strong for X/W	Strong for W (reused $512\times$), moderate for X (reused $32\times$)	W (16 KB) should live comfortably in L1; X (256 KB) should mostly live in L2/L3; associativity should matter wherever conflicting lines alias onto the same set.

Table 3: Predicted access-pattern properties, before running the simulator

5 Cache Hierarchy Configurations Tested

Three complete configurations were run, each keeping line size and capacities fixed and changing only associativity (the only variable the assignment permits):

Configuration	L1	L2	L3
A – Baseline	1-way	2-way	4-way
B – High assoc.	2-way	4-way	8-way
C – L1 focus	4-way	4-way	8-way

Table 4: Configurations tested

Rationale. Configuration A is the assignment’s own baseline and the reference point for everything else. Configuration B asks the broadest possible question – “if every level gets more ways, does the whole workload get faster?” – and is a fair, literal test of the hypothesis that the baseline’s associativity is too low across the board. Configuration C deliberately keeps L2 and L3 *identical to B* and changes only L1 (2-way $\rightarrow$ 4-way), so that the B-vs-C comparison isolates the marginal value of L1 associativity specifically, independent of any L2/L3 change – something the A-vs-B comparison cannot do cleanly, since A and B differ at all three levels simultaneously. This matters: several of the findings below (Section 8) depend on being able to tell whether an effect came from L1 or from L2/L3, and only the B-vs-C pair gives a clean answer to that.

6 Baseline Results

Table 5 below (auto-generated from the actual run) gives Configuration A’s phase-wise breakdown; the full three-way comparison follows in Section 7.

Phase	L1 Miss%	L2 Miss%	L3 Miss%	DRAM Accesses	Cycles	AMAT
Embedding	4.26	87.57	98.93	29,492	4,515,650	5.64
Matrix	50.41	1.64	4.32	12,005	125,117,428	3.72
Activation	100.00	6.25	90.42	118,517	27,056,052	12.90
Projection	1.76	7.24	100.00	5,376	5,227,203	1.24

Table 5: Phase-wise results, Configuration A_baseline

Phase	L1 Miss%	L2 Miss%	L3 Miss%	DRAM Accesses	Cycles	AMAT
Embedding	3.99	93.43	98.91	29,471	4,502,500	5.63
Matrix	50.31	2.46	2.89	11,995	127,710,088	3.80
Activation	6.25	100.00	90.45	118,552	17,229,152	8.22
Projection	0.13	98.17	100.00	5,376	4,883,188	1.16

Table 6: Phase-wise results, Configuration B_high_assoc

Phase	L1 Miss%	L2 Miss%	L3 Miss%	DRAM Accesses	Cycles	AMAT
Embedding	3.86	96.85	98.58	29,471	4,499,355	5.62
Matrix	50.31	2.46	2.89	11,995	127,708,968	3.80
Activation	6.25	100.00	90.45	118,553	17,229,252	8.22
Projection	0.13	100.00	100.00	5,376	4,882,688	1.16

Table 7: Phase-wise results, Configuration C_L1_focus

Config	Ways (L1/L2/L3)	L1 Miss%	L2 Miss%	L3 Miss%	DRAM Acc.	AMAT	Cycles
A_baseline	1/2/4	47.02	2.32	37.21	165,390	3.98	161,916,333
B_high_assoc	2/4/8	41.95	3.41	28.43	165,394	3.79	154,324,928
C_L1_focus	4/4/8	41.94	3.41	28.42	165,395	3.79	154,320,263

Table 8: Full-workload comparison across the three cache configurations

7 Experimental Results

Table 8 (above, from `tables.tex`) summarises the complete four-phase workload for all three configurations. The headline numbers:

- Total workload size is identical across configurations: 40,727,808 CPU accesses.
- DRAM accesses are essentially flat: 165,390 (A) vs. 165,394 (B) vs. 165,395 (C) – a difference of five accesses out of 165 thousand, i.e. noise.
- AMAT drops from 3.976 cycles/access (A) to 3.789 (B) and 3.789 (C) – a $\sim 4.7\%$ improvement, entirely from B onward; C adds essentially nothing over B (3.789178 $\rightarrow$ 3.789064, a 0.003% change).
- Total workload cycles: 161,916,333 (A) $\rightarrow$ 154,324,928 (B) $\rightarrow$ 154,320,263 (C).

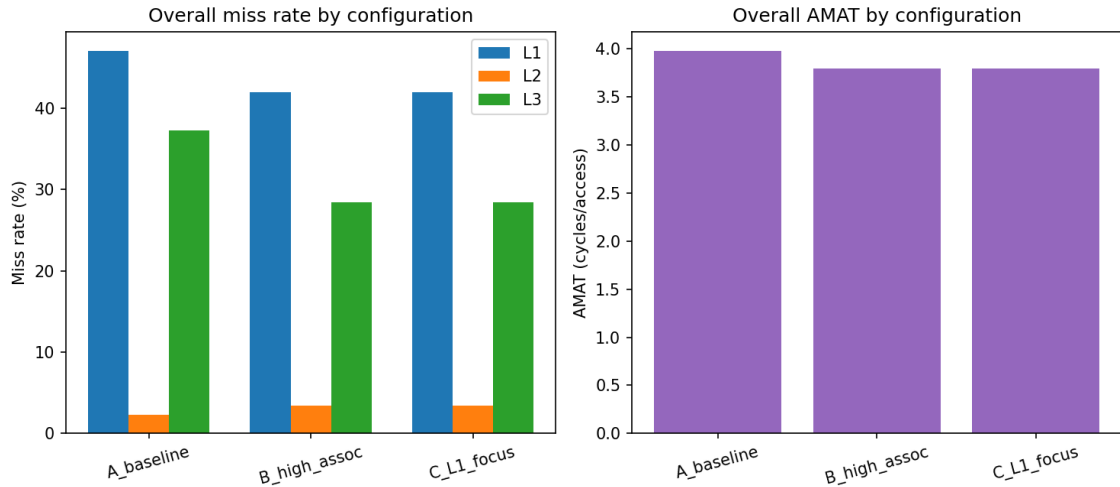


Figure 1: Overall miss rate by level (left) and overall AMAT (right), across the three configurations.

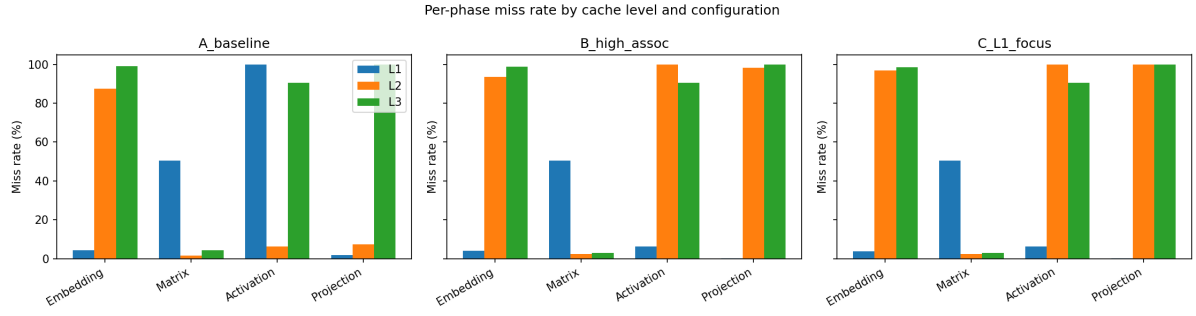


Figure 2: Per-phase, per-level miss rate for each configuration. Note how L2/L3 miss rate in Activation and Projection *rises* from A to B/C even as overall performance improves – discussed in Section 8.

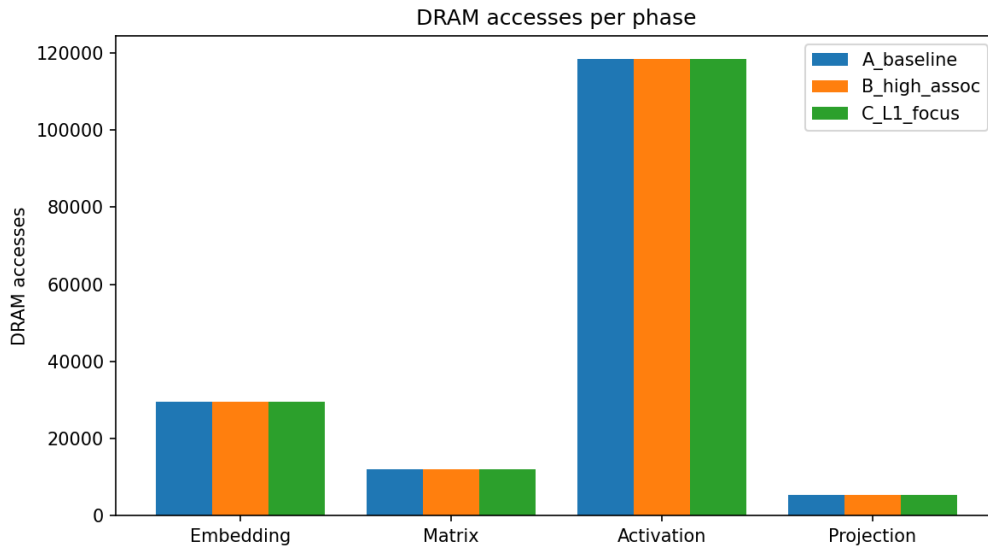


Figure 3: DRAM accesses per phase. Unlike the miss-rate figure above, this one is nearly flat across configurations within every phase – the true signal of whether a configuration change actually reduced expensive main-memory traffic.

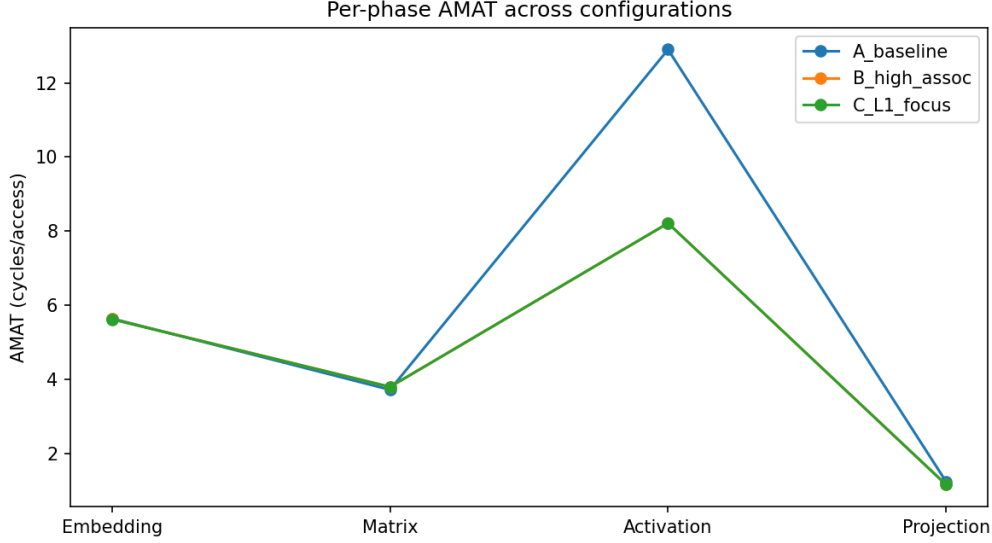


Figure 4: Per-phase AMAT across configurations.

8 Important Observations

8.1 Observation 1 – A genuine, quantifiable conflict-miss storm, fixed by 2-way L1

In the Activation phase, `input` is based at address 0 and `output` is based at address 4MB. 4MB is *exactly* $128\times$ the 32KB L1 size, so for every matching offset, the input line and the output line map to the *identical* L1 set. With a direct-mapped (1-way) L1, the interleaved `input[i]`, `output[i]` accesses evict each other on *every single access*: Configuration A shows an L1 miss rate of exactly 100.00% for this phase. Moving to a 2-way L1 (Configuration B) is sufficient to hold both aliasing lines in the same set simultaneously – L1 miss rate collapses to 6.25% (the theoretical floor: 1 compulsory miss per 16-word line, per stream). This is about as clean a demonstration of a conflict miss, and of associativity fixing it, as this kind of simulator can produce – it costs nothing in capacity (both lines still fit trivially in 2 ways) and the fix is complete after just one extra way.

8.2 Observation 2 – The same fix moves the miss burden *up*, not away

The obvious follow-on question: L2’s miss rate for the same phase was only 6.25% under the baseline – why does it jump to 100% once L1 becomes 2-way? Because L2 only ever sees what L1 forwards. Under the baseline, L1 misses on *every* access, so L2 sees the full 2,097,152-access stream and gets to exploit the “15 hits after 1 miss” pattern itself (131,072 misses out of 2,097,152 accesses = 6.25%). Once L1 absorbs that pattern (Observation 1), only the 131,072 genuinely first-ever-touched lines are forwarded to L2 at all – and because this phase is pure streaming with zero long-range reuse, *every one of those forwarded accesses is a compulsory miss at L2 too*: 131,072 misses out of 131,072 accesses = 100%. The same mechanism reproduces in Phase 4 (Projection): L2 accesses drop from 74,279 $\rightarrow$ 5,476 $\rightarrow$ 5,376 across A/B/C, converging exactly on 5,376 – the true compulsory-miss floor for that phase’s unique line count (4096 for X + 256 for W + 1024 for Y). **The number of L2 misses barely moves (roughly constant $\approx$ 131,072 / $\approx$ 5,376 across all three configurations) while L2’s own miss rate swings from single digits to 100%.** This is the report’s central methodological warning: a rising miss-rate percentage at an upper cache level, on its own, tells you nothing about whether the

hierarchy got worse – you have to look at the absolute traffic and the end-to-end DRAM/cycle counts together (Figure 3 vs. Figure 2).

8.3 Observation 3 – Increasing associativity does not fix a pure capacity problem

In the Matrix phase, L1 miss rate is essentially unchanged by associativity: 50.41% (A, 1-way) vs. 50.31% (B, 2-way) vs. 50.31% (C, 4-way) – a difference of a tenth of a percentage point. This is expected: B alone is 256 KB, eight times larger than the entire 32 KB L1, so no amount of associativity lets L1 retain useful fractions of B across the *i*-loop – this is a *capacity* miss, not a conflict miss, and associativity is architecturally the wrong tool for it. At L3 (2 MB, large enough to hold the whole 768 KB working set), associativity *does* help: L3 miss rate for Matrix falls from 4.32% (4-way) to 2.89% (8-way) – consistent with these now being genuine conflict misses that extra ways can resolve, because the data actually fits.

8.4 Observation 4 – A configuration that helps the whole workload can hurt one phase

Matrix-phase cycles alone are *worse* under Configuration B (127,710,088 cycles) than under the baseline (125,117,428 cycles) – a genuine regression of about 2.1%. L2 absolute miss count for Matrix rises sharply from A to B/C (278,208 $\rightarrow$ 415,603), and although L3 misses fall slightly (12,005 $\rightarrow$ 11,995), the extra accesses that now have to pay the full L1+L2+L3 latency chain (26 cycles) instead of stopping at L2 (6 cycles) outweigh that small L3 saving. If Matrix were evaluated in isolation, the baseline would be the right choice. But across the *complete* four-phase workload, B still wins decisively (154.3M vs. 161.9M total cycles) because Embedding, Activation, and Projection all improve enough to more than absorb Matrix’s regression. This is a direct, measured answer to the assignment’s own question: *a configuration that helps one phase is not necessarily best for the complete workload, and vice versa* – only a full-workload evaluation can tell you which effect dominates.

8.5 Observation 5 – Diminishing returns from L1 4-way over 2-way

Configuration C (L1 4-way) is, for every phase and overall, statistically indistinguishable from Configuration B (L1 2-way): overall AMAT differs by 0.003% (3.789178 vs. 3.789064), and overall DRAM accesses differ by exactly one access (165,394 vs. 165,395). Two extra L1 ways buy essentially nothing once the dominant conflict pattern (Observation 1, which only needed 2 ways to resolve) is already fixed – the remaining misses in every phase are capacity or compulsory misses that no amount of L1 associativity can touch.

9 Architectural Explanation of These Observations

Read together, Observations 1–5 trace exactly the causal chain the assignment asks for: *Workload Phase* $\rightarrow$ *Memory Access Pattern* $\rightarrow$ *Locality* $\rightarrow$ *Cache Mapping* $\rightarrow$ *Hit/Miss Behaviour* $\rightarrow$ *Level at which data is found* $\rightarrow$ *Memory Access Cost* $\rightarrow$ *Overall Performance*.

- The *access pattern* of a phase (streaming vs. strided vs. irregular-with-reuse) determines whether a miss, once it happens, is *fixable by associativity at all*. Activation’s input/output aliasing (Observation 1) is a pure conflict miss – associativity fixes it completely with 2 ways. Matrix’s L1 pressure from B (Observation 3) is a pure capacity miss – no amount of associativity fixes it; only a bigger L1 would.
- Because the hierarchy is strictly ordered (Section 2), whichever level first satisfies a request determines both its latency *and* which addresses the level above it will ever be asked to

serve. This is what produces Observation 2: making L1 better at absorbing local reuse literally changes L2’s input distribution, not just L2’s hit rate on a fixed distribution. Miss *rate* at an inner level is therefore not a stable, comparable quantity across configurations – miss *count* and downstream DRAM traffic are.

- Total workload performance is a sum over phases with very different access patterns, so a change that is a net win for the sum (B vs. A) can still be a net loss for an individual phase (Observation 4). Any hierarchy decision made by looking at one phase’s numbers in isolation risks exactly this trap.
- Returns from added associativity are not linear: the first extra way (1-way $\rightarrow$ 2-way at L1) resolved a 100%-miss pathology outright; the next two ways (2-way $\rightarrow$ 4-way) resolved essentially nothing further (Observation 5), because the remaining misses were never conflict misses to begin with.

10 Recommended Configuration

Level	Recommended associativity
L1	2-way
L2	4-way
L3	8-way

Table 9: Recommended hierarchy – Configuration B

This is *not* the configuration with the highest hit rate at any individual level – Configuration C has a marginally lower L1 miss rate than B at every phase, and neither B nor C has the highest L2 hit rate (that belongs to the baseline, Configuration A, for exactly the reasons in Observation 2). The recommendation is B specifically because:

1. It captures essentially all of the measured improvement over the baseline (161.9M $\rightarrow$ 154.3M total cycles, $\sim 4.7\%$ faster AMAT), driven almost entirely by eliminating the Activation-phase conflict-miss pathology in Observation 1 – a real, reproducible fix, not an artefact of the workload.
2. Pushing L1 to 4-way (Configuration C) is not worthwhile: it adds real hardware cost (twice the tag comparators and a wider mux at the level accessed on *every* cycle, where 1-cycle latency is hardest to hold) for a 0.003% AMAT change (Observation 5). This is squarely the kind of trade-off the assignment warns against rewarding on hit-rate grounds alone.
3. The Matrix-phase regression under B (Observation 4) is real but small in absolute terms (2.1% within that phase) and is more than paid for by the other three phases; a configuration chosen to avoid it (i.e. the baseline) would give up the much larger win elsewhere.

11 One Surprising Result

The most counter-intuitive result in this study is Observation 2: giving L1 more associativity *increased* L2’s own miss rate in two of the four phases – in Activation, from 6.25% to 100% – while DRAM traffic for that same phase barely moved (118,517 $\rightarrow$ 118,552 accesses) and overall performance improved. Before running this experiment, the intuitive expectation was that a “better” cache configuration should make miss rates go down at every level, or at worst stay flat. What it actually taught: a cache level’s miss rate is a measurement of *the traffic it happens*

to receive, not an absolute property of the workload – and in a strict inclusive-fill hierarchy, every level upstream can silently change what traffic a level downstream ever sees. A 100% miss rate at L2 sounds alarming in isolation, but here it was actually a symptom of L1 doing its job well (only forwarding accesses L1 genuinely could not have served), not evidence that L2 was failing. This is precisely why the assignment insists on reporting absolute DRAM accesses and AMAT alongside miss-rate percentages, rather than optimizing miss rate at any single level as an end in itself.

A Source Code

The complete, runnable, single-file source (`cache_sim.py`) is submitted alongside this report; running `python3 cache_sim.py` regenerates every table, CSV, JSON file, and plot referenced above from scratch (fixed random seed, so the numbers are exactly reproducible). It is reproduced in full below.

Listing 1: `cache_sim.py` – complete simulator, workload generators, experiment driver, and output writers

```

1  """
2  Exp-03: Multi-Level Cache Design -- single-file simulator.
3
4  Everything needed for the experiment lives in this one file:
5
6  1. A generic N-way set-associative LRU cache model ('Cache').
7  2. A three-level hierarchical cache ('MultiLevelCache') that enforces
8     strict CPU -> L1 -> L2 -> L3 -> DRAM lookup order, with data
9     filled back into every level above the one where it was found
10    (or from DRAM into all three, on a full miss).
11  3. Four workload-phase address-stream generators (embedding lookup,
12     matrix multiply, activation streaming, output projection) that
13     together model one four-phase "inference-like" workload.
14  4. An experiment driver that runs the complete four-phase workload,
15     WITHOUT clearing cache state between phases, through three
16     different associativity configurations, collecting per-phase and
17     per-configuration statistics.
18  5. Output: JSON + CSV result tables, matplotlib PNG plots, and a
19     LaTeX table fragment (results/tables.tex) that the report
20     \\input{s} directly, so every number in the PDF is reproduced
21     verbatim from an actual run of this script.
22
23  Run with: python3 cache_sim.py
24  System parameters (fixed by the assignment):
25      Address size      : 26 bits          -> 64 MB addressable
26      DRAM latency     : 100 cycles
27      Cache line size  : 64 B (all levels, fixed for the whole experiment)
28      L1 : 32 KB, baseline 1-way (direct mapped), 1 cycle
29      L2 : 256 KB, baseline 2-way,                    5 cycles
30      L3 : 2 MB, baseline 4-way,                        20 cycles
31  """
32
33  from __future__ import annotations
34
35  import csv
36  import json
37  import random
38  import time
39  from dataclasses import dataclass, field

```

```

40 from pathlib import Path
41
42 import matplotlib
43 matplotlib.use("Agg")
44 import matplotlib.pyplot as plt
45
46 # =====
47 # 1. System / cache-hierarchy constants (fixed for the whole experiment)
48 # =====
49
50 ADDRESS_BITS = 26
51 ADDRESS_SPACE_BYTES = 1 << ADDRESS_BITS          # 64 MB
52 DRAM_LATENCY = 100                                # cycles
53
54 LINE_SIZE = 64                                     # bytes, all levels
55
56 L1_SIZE = 32 * 1024
57 L2_SIZE = 256 * 1024
58 L3_SIZE = 2 * 1024 * 1024
59
60 L1_LATENCY = 1
61 L2_LATENCY = 5
62 L3_LATENCY = 20
63
64 RESULTS_DIR = Path(__file__).parent / "results"
65 PLOTS_DIR = RESULTS_DIR / "plots"
66 SEED = 42                                           # fixed, for reproducibility
67
68
69 # =====
70 # 2. Generic N-way set-associative LRU cache
71 # =====
72
73 class Cache:
74     """One N-way set-associative cache level with true LRU replacement.
75
76     'access(addr)' is a genuine demand access: it counts as a hit or a
77     miss and, on a miss, allocates the line in this cache (standard
78     allocate-on-miss policy).
79
80     'fill(addr)' inserts a line WITHOUT touching the hit/miss counters.
81     It models the "fill upper levels" step of the hierarchy diagram:
82     when a request is satisfied by a lower level (or by DRAM), the
83     block is written into this level as a side effect of that access,
84     not as a new demand access of its own.
85     """
86
87     __slots__ = (
88         "name", "ways", "num_sets", "set_mask", "offset_bits", "index_bits",
89         "latency", "tags", "lru", "clock", "hits", "misses",
90     )
91
92     def __init__(self, name: str, size_bytes: int, line_size: int, ways: int,
93                   latency: int):
94         self.name = name
95         self.ways = ways
96         num_lines = size_bytes // line_size
97         self.num_sets = num_lines // ways

```

```

97     assert self.num_sets & (self.num_sets - 1) == 0, "num_sets must be a power
of two"
98     self.set_mask = self.num_sets - 1
99     self.offset_bits = line_size.bit_length() - 1
100    self.index_bits = self.num_sets.bit_length() - 1
101    self.latency = latency
102
103    self.tags = [[-1] * ways for _ in range(self.num_sets)]
104    self.lru = [[0] * ways for _ in range(self.num_sets)]
105    self.clock = 0
106    self.hits = 0
107    self.misses = 0
108
109    def _insert(self, idx: int, tag: int) -> None:
110        set_tags = self.tags[idx]
111        set_lru = self.lru[idx]
112        victim = 0
113        min_stamp = set_lru[0]
114        if set_tags[0] == -1:
115            set_tags[0] = tag
116            set_lru[0] = self.clock
117            return
118        for w in range(1, self.ways):
119            t = set_tags[w]
120            if t == -1:
121                set_tags[w] = tag
122                set_lru[w] = self.clock
123                return
124            stamp = set_lru[w]
125            if stamp < min_stamp:
126                min_stamp = stamp
127                victim = w
128        set_tags[victim] = tag
129        set_lru[victim] = self.clock
130
131    def access(self, addr: int) -> bool:
132        self.clock += 1
133        line = addr >> self.offset_bits
134        idx = line & self.set_mask
135        tag = line >> self.index_bits
136        set_tags = self.tags[idx]
137        for w in range(self.ways):
138            if set_tags[w] == tag:
139                self.lru[idx][w] = self.clock
140                self.hits += 1
141                return True
142        self.misses += 1
143        self._insert(idx, tag)
144        return False
145
146    def fill(self, addr: int) -> None:
147        self.clock += 1
148        line = addr >> self.offset_bits
149        idx = line & self.set_mask
150        tag = line >> self.index_bits
151        for w in range(self.ways):
152            if self.tags[idx][w] == tag:
153                self.lru[idx][w] = self.clock

```

```

154         return
155         self._insert(idx, tag)
156
157     def snapshot(self) -> tuple[int, int]:
158         return self.hits, self.misses
159
160
161     # =====
162     # 3. Three-level hierarchical cache
163     # =====
164
165     class MultiLevelCache:
166         """CPU -> L1 -> L2 -> L3 -> DRAM, strictly in that order.
167
168         Latency model: cumulative / serial probing, i.e. an access that
169         misses in L1 and hits in L2 costs L1_LATENCY + L2_LATENCY cycles
170         (the L1 tag check is not free just because it missed). This is a
171         simplifying but standard assumption for a simulator of this kind;
172         it is stated explicitly here and in the report rather than left
173         implicit.
174         """
175
176         def __init__(self, l1_ways: int, l2_ways: int, l3_ways: int):
177             self.l1 = Cache("L1", L1_SIZE, LINE_SIZE, l1_ways, L1_LATENCY)
178             self.l2 = Cache("L2", L2_SIZE, LINE_SIZE, l2_ways, L2_LATENCY)
179             self.l3 = Cache("L3", L3_SIZE, LINE_SIZE, l3_ways, L3_LATENCY)
180             self.dram_accesses = 0
181             self.total_cycles = 0
182             self.total_accesses = 0
183
184         def access(self, addr: int) -> None:
185             self.total_accesses += 1
186             l1, l2, l3 = self.l1, self.l2, self.l3
187
188             if l1.access(addr):
189                 self.total_cycles += l1.latency
190                 return
191             if l2.access(addr):
192                 l1.fill(addr)
193                 self.total_cycles += l1.latency + l2.latency
194                 return
195             if l3.access(addr):
196                 l2.fill(addr)
197                 l1.fill(addr)
198                 self.total_cycles += l1.latency + l2.latency + l3.latency
199                 return
200             l3.fill(addr)
201             l2.fill(addr)
202             l1.fill(addr)
203             self.dram_accesses += 1
204             self.total_cycles += l1.latency + l2.latency + l3.latency + DRAM_LATENCY
205
206         def snapshot(self) -> dict:
207             return {
208                 "l1_hits": self.l1.hits, "l1_misses": self.l1.misses,
209                 "l2_hits": self.l2.hits, "l2_misses": self.l2.misses,
210                 "l3_hits": self.l3.hits, "l3_misses": self.l3.misses,
211                 "dram_accesses": self.dram_accesses,

```

```

212         "total_cycles": self.total_cycles,
213         "total_accesses": self.total_accesses,
214     }
215
216
217 # =====
218 # 4. Workload phases
219 #
220 # All four phases share the same 26-bit / 64 MB address space (that is
221 # the entire address space available on this system -- Phase 1's
222 # embedding table alone occupies all 64 MB of it, so Phases 2-4
223 # necessarily reuse addresses Phase 1 already touched, exactly as a
224 # real allocator would reuse heap space after the embedding lookup
225 # stage is done with it). Within each phase, arrays are placed at
226 # fixed, non-overlapping offsets so that phase-internal access
227 # patterns are unambiguous.
228 #
229 # Where a loop body naturally exposes a loop-invariant memory location
230 # to a compiler (e.g.  $C[i][j]$  across the  $k$ -loop, or  $Y[b][o]$  across the
231 #  $k$ -loop), we model the standard register-allocated behaviour: the
232 # accumulator lives in a register across the inner loop and is written
233 # back once. This keeps the address-stream size tractable without
234 # changing any of the spatial/strided/temporal properties the
235 # experiment cares about (those live entirely in the A/B/X/W traffic).
236 # =====
237
238 # ---- Phase 1: Embedding / Lookup -----
239 EMBED_ENTRIES = 1_048_576
240 EMBED_VALUES_PER_ENTRY = 16
241 EMBED_BYTES_PER_VALUE = 4
242 ENTRY_BYTES = EMBED_VALUES_PER_ENTRY * EMBED_BYTES_PER_VALUE # 64 B == 1 line
243 assert EMBED_ENTRIES * ENTRY_BYTES == ADDRESS_SPACE_BYTES
244
245 PHASE1_LOOKUPS = 50_000
246 PHASE1_HOT_POOL_SIZE = 64
247 PHASE1_HOT_PROB = 0.35
248 PHASE1_REPEAT_PROB = 0.05
249
250
251 def phase1_indices(rng: random.Random) -> list[int]:
252     """Irregular, non-sequential index stream with weak spatial locality
253     between lookups, but deliberate temporal reuse via a small 'hot'
254     pool of indices and occasional immediate repeats -- e.g. the kind
255     of pattern shown in the spec: 17, 2048, 91, 17, 8192, 2048, 503, 91..."""
256     hot_pool = rng.sample(range(EMBED_ENTRIES), PHASE1_HOT_POOL_SIZE)
257     seq = []
258     prev = None
259     for _ in range(PHASE1_LOOKUPS):
260         r = rng.random()
261         if prev is not None and r < PHASE1_REPEAT_PROB:
262             idx = prev
263         elif r < PHASE1_REPEAT_PROB + PHASE1_HOT_PROB:
264             idx = rng.choice(hot_pool)
265         else:
266             idx = rng.randrange(EMBED_ENTRIES)
267         seq.append(idx)
268         prev = idx
269     return seq

```

```

270
271
272 def phase1_addresses(rng: random.Random):
273     for idx in phase1_indices(rng):
274         base = idx * ENTRY_BYTES
275         for v in range(EMBED_VALUES_PER_ENTRY):
276             yield base + v * EMBED_BYTES_PER_VALUE
277
278
279 # ---- Phase 2: Matrix / Tensor Computation ( $C = A \times B$ , ijk order) -----
280 MAT_N = 256
281 MAT_A_BASE = 0
282 MAT_B_BASE = MAT_N * MAT_N * 4
283 MAT_C_BASE = 2 * MAT_N * MAT_N * 4
284
285
286 def phase2_addresses():
287     N = MAT_N
288     a_base, b_base, c_base = MAT_A_BASE, MAT_B_BASE, MAT_C_BASE
289     for i in range(N):
290         iN = i * N
291         for j in range(N):
292             for k in range(N):
293                 yield a_base + (iN + k) * 4          #  $A[i][k]$ : sequential in k
294                 yield b_base + (k * N + j) * 4        #  $B[k][j]$ : strided, stride = N
295                 *4
296                 yield c_base + (iN + j) * 4          #  $C[i][j]$ : one write-back per (
297                 i, j)
298
299 # ---- Phase 3: Activation Processing (streaming) -----
300 ACT_N = 1_048_576
301 ACT_INPUT_BASE = 0
302 ACT_OUTPUT_BASE = ACT_N * 4
303
304 def phase3_addresses():
305     in_base, out_base = ACT_INPUT_BASE, ACT_OUTPUT_BASE
306     for i in range(ACT_N):
307         off = i * 4
308         yield in_base + off
309         yield out_base + off
310
311
312 # ---- Phase 4: Output / Projection ( $Y = X \cdot W$ , batched) -----
313 PROJ_BATCH = 512
314 PROJ_IN = 128
315 PROJ_OUT = 32
316 PROJ_X_BASE = 0
317 PROJ_W_BASE = PROJ_BATCH * PROJ_IN * 4
318 PROJ_Y_BASE = PROJ_W_BASE + PROJ_IN * PROJ_OUT * 4
319
320
321 def phase4_addresses():
322     B, IN, OUT = PROJ_BATCH, PROJ_IN, PROJ_OUT
323     x_base, w_base, y_base = PROJ_X_BASE, PROJ_W_BASE, PROJ_Y_BASE
324     for b in range(B):
325         bIN = b * IN

```



```

326         for o in range(OUT):
327             oIN = o * IN
328             for k in range(IN):
329                 yield x_base + (bIN + k) * 4          # X[b][k]: sequential, reused
    across o
330                 yield w_base + (oIN + k) * 4          # W[o][k]: sequential, reused
    across b
331                 yield y_base + (b * OUT + o) * 4      # Y[b][o]: strided write-back
332
333
334 PHASES = [
335     ("Embedding", phase1_addresses, True),    # True => generator needs rng
336     ("Matrix", phase2_addresses, False),
337     ("Activation", phase3_addresses, False),
338     ("Projection", phase4_addresses, False),
339 ]
340
341
342 # =====
343 # 5. Experiment driver
344 # =====
345
346 CONFIGS = [
347     # (label, l1_ways, l2_ways, l3_ways, rationale)
348     ("A_baseline", 1, 2, 4,
349      "Assignment baseline: L1 direct-mapped, L2 2-way, L3 4-way."),
350     ("B_high_assoc", 2, 4, 8,
351      "Raise associativity at every level relative to baseline, to test "
352      "whether conflict misses observed in A are actually reduced when "
353      "every level gets more ways."),
354     ("C_L1_focus", 4, 4, 8,
355      "Same L2/L3 as B, but push L1 to 4-way. Isolates whether L1 "
356      "associativity specifically matters, given L1 is by far the "
357      "highest-traffic, lowest-latency, smallest level."),
358 ]
359
360
361 def pct(n: int, d: int) -> float:
362     return 100.0 * n / d if d else 0.0
363
364
365 def run_config(label: str, l1w: int, l2w: int, l3w: int) -> dict:
366     mlc = MultiLevelCache(l1w, l2w, l3w)
367     rng = random.Random(SEED)
368
369     phase_rows = []
370     prev_snap = mlc.snapshot()
371
372     for phase_name, gen_fn, needs_rng in PHASES:
373         t0 = time.perf_counter()
374         gen = gen_fn(rng) if needs_rng else gen_fn()
375         access = mlc.access
376         for addr in gen:
377             access(addr)
378         elapsed = time.perf_counter() - t0
379
380         snap = mlc.snapshot()
381         d = {k: snap[k] - prev_snap[k] for k in snap}

```

```

382     prev_snap = snap
383
384     l1_acc = d["l1_hits"] + d["l1_misses"]
385     l2_acc = d["l2_hits"] + d["l2_misses"]
386     l3_acc = d["l3_hits"] + d["l3_misses"]
387     phase_rows.append({
388         "phase": phase_name,
389         "l1_accesses": l1_acc, "l1_hits": d["l1_hits"], "l1_misses": d["
l1_misses"],
390         "l2_accesses": l2_acc, "l2_hits": d["l2_hits"], "l2_misses": d["
l2_misses"],
391         "l3_accesses": l3_acc, "l3_hits": d["l3_hits"], "l3_misses": d["
l3_misses"],
392         "l1_miss_pct": pct(d["l1_misses"], l1_acc),
393         "l2_miss_pct": pct(d["l2_misses"], l2_acc),
394         "l3_miss_pct": pct(d["l3_misses"], l3_acc),
395         "dram_accesses": d["dram_accesses"],
396         "cycles": d["total_cycles"],
397         "cpu_accesses": d["total_accesses"],
398         "amat": d["total_cycles"] / d["total_accesses"] if d["total_accesses"]
else 0.0,
399         "wall_seconds": elapsed,
400     })
401     print(f"    [{label}] phase {phase_name:<10s} "
402           f"accesses={d['total_accesses']:>10,d} "
403           f"L1miss%={phase_rows[-1]['l1_miss_pct']:5.2f} "
404           f"L2miss%={phase_rows[-1]['l2_miss_pct']:5.2f} "
405           f"L3miss%={phase_rows[-1]['l3_miss_pct']:5.2f} "
406           f"DRAM={d['dram_accesses']:>9,d} ({elapsed:5.2f}s)")
407
408     overall = mlc.snapshot()
409     l1_acc = overall["l1_hits"] + overall["l1_misses"]
410     l2_acc = overall["l2_hits"] + overall["l2_misses"]
411     l3_acc = overall["l3_hits"] + overall["l3_misses"]
412     summary = {
413         "config": label,
414         "l1_ways": l1w, "l2_ways": l2w, "l3_ways": l3w,
415         "l1_miss_pct": pct(overall["l1_misses"], l1_acc),
416         "l2_miss_pct": pct(overall["l2_misses"], l2_acc),
417         "l3_miss_pct": pct(overall["l3_misses"], l3_acc),
418         "dram_accesses": overall["dram_accesses"],
419         "cpu_accesses": overall["total_accesses"],
420         "total_cycles": overall["total_cycles"],
421         "amat": overall["total_cycles"] / overall["total_accesses"] if overall["
total_accesses"] else 0.0,
422     }
423     return {"summary": summary, "phases": phase_rows}
424
425
426 def main() -> None:
427     RESULTS_DIR.mkdir(parents=True, exist_ok=True)
428     PLOTS_DIR.mkdir(parents=True, exist_ok=True)
429
430     all_results = []
431     for label, l1w, l2w, l3w, rationale in CONFIGS:
432         print(f"\n=== Running configuration {label} "
433               f"(L1={l1w}-way, L2={l2w}-way, L3={l3w}-way) ===")
434         print(f"    rationale: {rationale}")

```

```

435     res = run_config(label, l1w, l2w, l3w)
436     res["rationale"] = rationale
437     all_results.append(res)
438
439     write_json(all_results)
440     write_phase_csv(all_results)
441     write_config_csv(all_results)
442     append_master_ledger(all_results)
443     write_latex_tables(all_results)
444     make_plots(all_results)
445
446     print(f"\nAll done. Results written to: {RESULTS_DIR}")
447
448
449     # =====
450     # 6. Output: JSON / CSV / master ledger / LaTeX tables / plots
451     # =====
452
453     def write_json(all_results: list[dict]) -> None:
454         with open(RESULTS_DIR / "raw_results.json", "w") as f:
455             json.dump(all_results, f, indent=2)
456
457
458     def write_phase_csv(all_results: list[dict]) -> None:
459         path = RESULTS_DIR / "phase_wise_results.csv"
460         fields = ["config", "phase", "l1_miss_pct", "l2_miss_pct", "l3_miss_pct",
461                  "dram_accesses", "cycles", "cpu_accesses", "amat", "wall_seconds"]
462         with open(path, "w", newline="") as f:
463             w = csv.DictWriter(f, fieldnames=fields)
464             w.writeheader()
465             for res in all_results:
466                 cfg = res["summary"]["config"]
467                 for row in res["phases"]:
468                     w.writerow({"config": cfg, **{k: row[k] for k in fields if k != "
469                                     config"}}})
470
471
472     def write_config_csv(all_results: list[dict]) -> None:
473         path = RESULTS_DIR / "config_summary.csv"
474         fields = ["config", "l1_ways", "l2_ways", "l3_ways", "l1_miss_pct", "
475                  l2_miss_pct",
476                  "l3_miss_pct", "dram_accesses", "cpu_accesses", "total_cycles", "amat
477                  "]
478         with open(path, "w", newline="") as f:
479             w = csv.DictWriter(f, fieldnames=fields)
480             w.writeheader()
481             for res in all_results:
482                 w.writerow(res["summary"])
483
484
485     def append_master_ledger(all_results: list[dict]) -> None:
486         """Append-only run log: every run (config x phase) with a timestamp,
487         never overwritten -- so successive script runs accumulate history
488         instead of destroying it."""
489         path = RESULTS_DIR / "master_ledger.csv"
490         fields = ["timestamp", "config", "l1_ways", "l2_ways", "l3_ways", "phase",
491                  "l1_miss_pct", "l2_miss_pct", "l3_miss_pct", "dram_accesses",
492                  "cycles", "amat", "notes"]

```

```

490 write_header = not path.exists()
491 ts = time.strftime("%Y-%m-%d %H:%M:%S")
492 with open(path, "a", newline="") as f:
493     w = csv.DictWriter(f, fieldnames=fields)
494     if write_header:
495         w.writeheader()
496     for res in all_results:
497         s = res["summary"]
498         for row in res["phases"]:
499             w.writerow({
500                 "timestamp": ts, "config": s["config"],
501                 "l1_ways": s["l1_ways"], "l2_ways": s["l2_ways"], "l3_ways": s[
502 "l3_ways"],
503                 "phase": row["phase"], "l1_miss_pct": f"{row['l1_miss_pct']:.3f}
504 }",
505                 "l2_miss_pct": f"{row['l2_miss_pct']:.3f}", "l3_miss_pct": f"{
506 row['l3_miss_pct']:.3f}",
507                 "dram_accesses": row["dram_accesses"], "cycles": row["cycles"],
508                 "amat": f"{row['amat']:.3f}", "notes": res["rationale"],
509             })
510     w.writerow({
511         "timestamp": ts, "config": s["config"],
512         "l1_ways": s["l1_ways"], "l2_ways": s["l2_ways"], "l3_ways": s["
513 l3_ways"],
514         "phase": "OVERALL", "l1_miss_pct": f"{s['l1_miss_pct']:.3f}",
515         "l2_miss_pct": f"{s['l2_miss_pct']:.3f}", "l3_miss_pct": f"{s['
516 l3_miss_pct']:.3f}",
517         "dram_accesses": s["dram_accesses"], "cycles": s["total_cycles"],
518         "amat": f"{s['amat']:.3f}", "notes": res["rationale"],
519     })
520
521 def write_latex_tables(all_results: list[dict]) -> None:
522     """Emit results/tables.tex: auto-generated LaTeX tables reproduced
523     directly from this run's numbers, so the report never hand-transcribes
524     a figure. \\input{} this file from the report."""
525     lines = []
526
527     # Table: phase-wise results (one sub-table per configuration)
528     for res in all_results:
529         cfg = res["summary"]["config"]
530         lines.append(r"\begin{table}[H]")
531         lines.append(r"\centering")
532         lines.append(r"\begin{tabular}{lrrrrrr}")
533         lines.append(r"\toprule")
534         lines.append(r"Phase & L1 Miss\% & L2 Miss\% & L3 Miss\% & DRAM Accesses &
535 Cycles & AMAT \\")
536         lines.append(r"\midrule")
537         for row in res["phases"]:
538             lines.append(
539                 f"{row['phase']} & {row['l1_miss_pct']:.2f} & {row['l2_miss_pct']:.2f} & "
540                 f"{row['l3_miss_pct']:.2f} & {row['dram_accesses'],} & {row['
541 cycles'],} & "
542                 f"{row['amat']:.2f} \\\\"
543             )
544         lines.append(r"\bottomrule")
545         lines.append(r"\end{tabular}")

```

```

540     cfg_esc = cfg.replace("_", r"\_")
541     lines.append(rf"\caption{{Phase-wise results, Configuration {cfg_esc}}}")
542     lines.append(rf"\label{{tab:phase-{cfg}}}")
543     lines.append(r"\end{table}")
544     lines.append("")
545
546     # Table: configuration comparison
547     lines.append(r"\begin{table}[H]")
548     lines.append(r"\centering")
549     lines.append(r"\small")
550     lines.append(r"\begin{tabular}{lcrrrrrr}")
551     lines.append(r"\toprule")
552     lines.append(r"Config & Ways (L1/L2/L3) & L1 Miss\% & L2 Miss\% & L3 Miss\% & DRAM Acc. & AMAT & Cycles \\")
553     lines.append(r"\midrule")
554     for res in all_results:
555         s = res["summary"]
556         cfg_esc = s["config"].replace("_", r"\_")
557         lines.append(
558             f"{cfg_esc} & {s['l1_ways']}/{s['l2_ways']}/{s['l3_ways']} & "
559             f"{s['l1_miss_pct']:.2f} & {s['l2_miss_pct']:.2f} & {s['l3_miss_pct']:.2f} & "
560             f"{s['dram_accesses'],} & {s['amat']:.2f} & {s['total_cycles'],} \\\\"
561             )
562     lines.append(r"\bottomrule")
563     lines.append(r"\end{tabular}")
564     lines.append(r"\caption{Full-workload comparison across the three cache configurations}")
565     lines.append(r"\label{tab:config-comparison}")
566     lines.append(r"\end{table}")
567
568     with open(RESULTS_DIR / "tables.tex", "w") as f:
569         f.write("\n".join(lines) + "\n")
570
571
572 def make_plots(all_results: list[dict]) -> None:
573     configs = [r["summary"]["config"] for r in all_results]
574     phases = [row["phase"] for row in all_results[0]["phases"]]
575
576     # --- Plot 1: per-phase, per-level miss rate, one subplot per config ---
577     fig, axes = plt.subplots(1, len(all_results), figsize=(5 * len(all_results), 4),
578                             , sharey=True)
579     if len(all_results) == 1:
580         axes = [axes]
581     x = range(len(phases))
582     width = 0.25
583     for ax, res in zip(axes, all_results):
584         l1 = [row["l1_miss_pct"] for row in res["phases"]]
585         l2 = [row["l2_miss_pct"] for row in res["phases"]]
586         l3 = [row["l3_miss_pct"] for row in res["phases"]]
587         ax.bar([i - width for i in x], l1, width, label="L1")
588         ax.bar(list(x), l2, width, label="L2")
589         ax.bar([i + width for i in x], l3, width, label="L3")
590         ax.set_xticks(list(x))
591         ax.set_xticklabels(phases, rotation=30, ha="right")
592         ax.set_title(res["summary"]["config"])
593         ax.set_ylabel("Miss rate (%)")

```

```

593 axes[0].legend()
594 fig.suptitle("Per-phase miss rate by cache level and configuration")
595 fig.tight_layout()
596 fig.savefig(PLOTS_DIR / "phase_miss_rates.png", dpi=150)
597 plt.close(fig)
598
599 # --- Plot 2: DRAM accesses per phase per config ---
600 fig, ax = plt.subplots(figsize=(8, 4.5))
601 width = 0.8 / len(all_results)
602 for ci, res in enumerate(all_results):
603     vals = [row["dram_accesses"] for row in res["phases"]]
604     offs = [i + (ci - (len(all_results) - 1) / 2) * width for i in range(len(
        phases))]
605     ax.bar(offs, vals, width, label=res["summary"]["config"])
606 ax.set_xticks(range(len(phases)))
607 ax.set_xticklabels(phases)
608 ax.set_ylabel("DRAM accesses")
609 ax.set_title("DRAM accesses per phase")
610 ax.legend()
611 fig.tight_layout()
612 fig.savefig(PLOTS_DIR / "dram_accesses_per_phase.png", dpi=150)
613 plt.close(fig)
614
615 # --- Plot 3: AMAT per phase per config ---
616 fig, ax = plt.subplots(figsize=(8, 4.5))
617 for res in all_results:
618     vals = [row["amat"] for row in res["phases"]]
619     ax.plot(phases, vals, marker="o", label=res["summary"]["config"])
620 ax.set_ylabel("AMAT (cycles/access)")
621 ax.set_title("Per-phase AMAT across configurations")
622 ax.legend()
623 fig.tight_layout()
624 fig.savefig(PLOTS_DIR / "amat_per_phase.png", dpi=150)
625 plt.close(fig)
626
627 # --- Plot 4: overall config comparison (miss rates + AMAT) ---
628 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.5))
629 l1 = [r["summary"]["l1_miss_pct"] for r in all_results]
630 l2 = [r["summary"]["l2_miss_pct"] for r in all_results]
631 l3 = [r["summary"]["l3_miss_pct"] for r in all_results]
632 xs = range(len(configs))
633 width = 0.25
634 ax1.bar([i - width for i in xs], l1, width, label="L1")
635 ax1.bar(list(xs), l2, width, label="L2")
636 ax1.bar([i + width for i in xs], l3, width, label="L3")
637 ax1.set_xticks(list(xs))
638 ax1.set_xticklabels(configs, rotation=15)
639 ax1.set_ylabel("Miss rate (%)")
640 ax1.set_title("Overall miss rate by configuration")
641 ax1.legend()
642
643 amat = [r["summary"]["amat"] for r in all_results]
644 ax2.bar(configs, amat, color="tab:purple")
645 ax2.set_ylabel("AMAT (cycles/access)")
646 ax2.set_title("Overall AMAT by configuration")
647 ax2.set_xticklabels(configs, rotation=15)
648 fig.tight_layout()
649 fig.savefig(PLOTS_DIR / "config_comparison.png", dpi=150)

```

```
650     plt.close(fig)
651
652
653 if __name__ == "__main__":
654     main()
```